

Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey.

Inteligencia Artificial Avanzada para la Ciencia de Datos
“Reto CAETEC | Reporte de Técnicas de Modelado”

Profesores:
Benjamín Valdés Aguirre
Eduardo Daniel Juárez Pineda

Jorge Armando Garzón Varela | A01707102
Santiago Alducin Villaseñor | A01707122
Sebastián Pérez García | A01707346
Roberto Serna Niño | A01709577
Kamila Jeannette Martinez Ibarra | A01711833

Fecha de actualización: 3 de noviembre de 2026

1. Introducción

Para la fase de modelado se investigaron cinco técnicas candidatas para estudiar la formación de colas en los tres robots de ordeño (VMS) del CAETEC: LSTM, TCN, GRU, MLP Classifier y ARIMA/SARIMA. Este reporte reúne lo que se encontró de cada una, cómo se aplicaría a nuestros datos y qué decisiones quedan por alinear antes de entrenar. A continuación se presenta cada técnica con lo investigado en su pestaña correspondiente, seguida de una comparación entre ellas.

2. Variable objetivo y datasets

Todas las técnicas parten de la variable pct_posible_cola: la proporción de ordeños de un bloque de 30 minutos cuyo hueco hasta el siguiente ordeño del mismo robot es de 0 a 2 minutos. Data Preparation dejó dos datasets con esta variable:

- Dataset de regresión (dataset_final_regresion.parquet): 132,028 bloques observados y 22 columnas, con huecos entre bloques y tres variables del bloque anterior ya calculadas. Sirve para los modelos que tratan cada bloque como una fila independiente.
- Dataset de serie de tiempo (dataset_final_serie_tiempo.parquet): 179,850 bloques continuos de 30 minutos por robot y 24 columnas, incluidos 47,792 bloques sin ordeños. Es el que necesitan los modelos secuenciales, porque conserva la malla temporal.

Las variables mecánicas del mismo bloque (total_ordenios, duracion_total_min, pct_ocupacion_robot y ordenios_otros_robots) definen la cola por construcción, por lo que ninguna técnica debe usar las del bloque que se quiere predecir.

3. Modelo LSTM

3.1 Fundamentos de la arquitectura

LSTM es una capa secuencial que recibe como entrada el contexto a largo plazo y el resultado de la entrada anterior, y mediante una serie de operaciones y compuertas determina

qué y cómo se recuerdan las cosas. Esto ayuda mucho en las series de tiempo, ya que permite al modelo tener en consideración predicciones pasadas y la secuencia de datos al entrenar para generar mejores predicciones. Debido a que estamos trabajando con la formación de las colas, la utilización de datos históricos para influir en la predicción puede ser de mucha utilidad.

3.2 LSTM en predicción de colas en semáforos

Long Short-Term Memory es un algoritmo de deep learning que se ha utilizado para comprender la formación de las colas en semáforos en el proyecto de Rahman y Hasan (2021). En este generan un sistema en tiempo real capaz de determinar el tamaño de la cola en los semáforos, con el propósito de disminuir el tiempo en estos y poder adaptar los tiempos de semáforo de manera dinámica. El estudio utiliza únicamente la pérdida como métrica para evaluar el modelo, en el cual reporta una pérdida (RMSE y MAE) similar o inferior a 1.

La arquitectura que eligieron consistía en dos capas de LSTM con ReLU y el optimizador Adam. Al compararlo con otros modelos como KNN, FFNN, RNN y SVR, la LSTM tuvo mejores resultados y menor error en la mayoría de los casos, con muy poca diferencia frente a la RNN y la FFNN.

3.3 LSTM para la predicción de actividad de las vacas

En un ejemplo más cercano al proyecto actual, Ahmed, Malick y Al-Shamayleh (2026) generan un modelo para detectar la actividad de las vacas utilizando LSTM y SMOTE (Synthetic Minority Over-sampling Technique), con el que predicen la actividad de las vacas con un F1 de 97.29 %. Una diferencia significativa de esta aplicación es el uso de SMOTE para entrenar el modelo en casos poco habituales. La arquitectura presenta 3 capas de LSTM seguidas de un par de capas densas para la categorización final de la actividad que está haciendo la vaca:

- La primera capa son 128 unidades con return_sequences=True, que facilita la retención del contexto temporal.
- La segunda capa son 64 unidades con return_sequences=True, que aumenta la retención del contexto temporal.

- La última LSTM solo tiene 32 unidades con return_sequences=False, colapsando el modelo a un solo vector. Este ya captura los detalles temporales y los condensa para el análisis de la capa densa.
- Para evitar el sobreajuste integra una capa de Dropout al 50 %, y las últimas capas densas utilizan ReLU para la generalización e identificación de patrones complejos y Softmax para la categorización, respectivamente.
- Se utiliza SparseCategoricalCrossEntropy como función de pérdida.

3.4 Conclusiones sobre la utilización de LSTM para el proyecto

Estos dos artículos proponen evidencia del uso de una red LSTM para reconocer patrones de colas y de comportamiento de las vacas, y el objetivo del proyecto sería evaluar si se puede aplicar una estrategia y arquitectura similar para la identificación de patrones en la formación de colas de ordeño.

3.5 Precauciones

- Debido a la naturaleza secuencial del modelo, se debe contar con datos continuos para el entrenamiento y las predicciones.
- Las aplicaciones analizadas fueron en un contexto y con datos diferentes. Que haya funcionado con el movimiento de las vacas y con la formación de colas en tráfico no asegura que vaya a servir para la predicción de colas de ordeño con los datos que tenemos.

4. Modelo TCN (Temporal Convolutional Network)

4.1 Fundamentos de la arquitectura

Una TCN no es una arquitectura única sino una familia de redes. Está definida como una red totalmente convolucional 1D con convoluciones causales: la salida en el tiempo t solo depende de entradas del tiempo t y anteriores, y la salida conserva la longitud de la entrada. Para mirar historia larga combinan convoluciones dilatadas, con dilatación exponencial (d = 2^i), y bloques residuales con dos capas de convolución causal dilatada, ReLU, weight normalization y dropout espacial.

Al compararla con LSTM, GRU y RNN en tareas como el adding problem, MNIST secuencial, memoria de copia, música polifónica y modelado de lenguaje, la TCN genérica

superó a las redes recurrentes en la mayoría, con una memoria efectiva más larga (Bai et al., 2018). No siempre es así: en Penn Treebank a nivel palabra, una LSTM optimizada superó a la TCN.

Las ventajas son el paralelismo, los gradientes estables y el bajo consumo de memoria en entrenamiento. Las desventajas son que en evaluación necesita la secuencia cruda hasta la longitud de su campo receptivo y que al cambiar de dominio puede requerir reajustar el tamaño del filtro y las dilataciones.

4.2 TCN en predicción de tráfico y congestión vehicular

Zhao et al. (2019) proponen un predictor de flujo vehicular a corto plazo basado en TCN, con el fin de anticipar posibles sitios de congestión. Usaron conteos de vehículos cada 15 minutos en una carretera de Birmingham (Reino Unido) durante 2014, con 35,040 registros. Los primeros 9 meses fueron para entrenamiento y el resto para prueba, y cada ventana de cinco valores pasados predice el siguiente.

Para definir la arquitectura emplearon el método Taguchi con un arreglo ortogonal L16, en lugar de prueba y error. La configuración final tuvo 24 filtros, 6 pilas de bloques residuales, kernel de 15 y dilataciones de 2^0 a 2^9. Con esa configuración la TCN obtuvo un MAE de 8.43 y un MRE de 0.0458, frente a un MAE de 29.61 y un MRE de 0.1964 de la LSTM. El MAE de la GRU fue de 36.09. Los autores reportan una exactitud cercana al 95 % y una predicción en menos de un segundo. Reconocen que el modelo falla en días atípicos (clima o festivos), porque hay pocos ejemplos de esos eventos para entrenar.

4.3 TCN para el comportamiento de las vacas

En un ejemplo más cercano al proyecto, aunque no sobre colas, Wei et al. (2023) clasifican la pose de vacas (de pie, caminando, acostada) con una Multi-Scale TCN. La entrada son las coordenadas de 16 puntos del esqueleto de la vaca en cuadros consecutivos de video, y la red se compone de cuatro módulos SS-TCN con convoluciones dilatadas.

La arquitectura termina con un mecanismo de atención antes de una capa densa de 3 neuronas con softmax. Entrenaron con entropía cruzada categórica, Adam (tasa de 0.001), 300 épocas y dropout de 0.3. La variante con Coordinate Attention y activación GELU alcanzó una

precisión de 94.71 %, un recall de 86.99 % y un F1 de 90.69 %. La MS-TCN sin atención, con ReLU y al mismo umbral, tuvo un F1 de 73.17 %. La mejora vino de añadir atención y cambiar la activación. El conjunto de datos es pequeño: 1,800 secuencias de 9 vacas, con 420 secuencias de prueba.

4.4 Utilización

Estos trabajos sostienen que una TCN es una candidata razonable para modelar la dinámica temporal de colas, con algunos detalles. Su diseño encaja con los datasets finales que se armaron:

- Malla regular: el dataset de serie de tiempo (179,850 bloques de 30 minutos por robot) conserva una malla continua, que es lo que necesita una convolución 1D.
- Causalidad: la TCN responde a la preocupación de fuga de información ya documentada. Las variables mecánicas del mismo bloque (total_ordenios, duracion_total_min, pct_ocupacion_robot, ordenios_otros_robots) definen la cola por construcción. La ventana de entrada debe terminar en el bloque t−1 y la etiqueta ubicarse en t.
- Campo receptivo (cálculo propio): con la fórmula de Bai et al., RF = 1 + 2(k−1)(2^n − 1) para bloques de dos convoluciones. Con k = 3 y n = 4 da 61 bloques (≈30.5 h), que cubre un día completo y los horarios de alimentación de 7:30 y 14:30. Con n = 6 son 253 bloques (≈5.3 días) y con n = 7 son 509 (≈10.6 días).

4.5 Precauciones

- Interpretabilidad: la TCN es poco interpretable. Cubre el criterio de capacidad predictiva, pero la etapa de lactancia más asociada (chi-cuadrado, $p < 0.05$) requiere pruebas estadísticas aparte.
- Línea base: los resultados de Zhao et al. vienen de un solo conjunto de datos, y Bai et al. muestran que una LSTM bien optimizada puede ganar en corpus pequeños. Conviene comparar contra la LSTM ya investigada y contra una línea base.
- Sin precedente directo: no se encontraron estudios con TCN aplicados a colas de robots de ordeño. El trabajo de Wei et al. sirve como referencia de diseño, no de desempeño esperado.

5. Modelo GRU (Gated Recurrent Unit)

5.1 Fundamentos de la arquitectura

La GRU fue propuesta por Cho et al. (2014) dentro de un modelo encoder–decoder para traducción automática, como una versión simplificada de la LSTM. Al igual que la LSTM, usa compuertas para decidir qué información conservar y cuál descartar, lo que reduce el problema de desvanecimiento del gradiente de la RNN simple. A diferencia de la LSTM, no tiene una celda de memoria separada: el estado oculto h hace las dos funciones.

Tiene dos compuertas. La compuerta de actualización (z) decide cuánto del estado anterior se conserva y cuánto se reemplaza; combina en una sola las compuertas de olvido y de entrada de la LSTM. La compuerta de reinicio (r) decide cuánto del estado anterior se usa para calcular el estado candidato. Las ecuaciones son:

- $z_t = \sigma(W_z \cdot x_t + U_z \cdot h_{t-1} + b_z)$
- $r_t = \sigma(W_r \cdot x_t + U_r \cdot h_{t-1} + b_r)$
- $\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}) + b_h)$
- $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

Al tener tres bloques de pesos en lugar de cuatro, una GRU tiene aproximadamente tres cuartas partes de los parámetros de una LSTM del mismo tamaño, por lo que entrena más rápido y necesita menos datos para no sobreajustar. Chung et al. (2014), quienes le dieron el nombre de GRU, la compararon con la LSTM y la RNN simple en modelado de música y de voz: GRU y LSTM tuvieron un desempeño parecido, ambas muy superiores a la RNN simple, y ninguna fue mejor en todos los casos.

5.2 GRU en predicción de flujos y llegadas

No se encontró un estudio de GRU aplicado directamente a colas, pero sí a problemas de llegadas a un servicio, que es lo que pasa cuando las vacas llegan al robot. Das (2023) compara LSTM y GRU para predecir el volumen de tráfico por hora en la carretera I-94 (Minneapolis), con 47,284 registros de 2012 a 2018. Entrenó con 2012–2017 y probó con el último año, escalando con MinMax. Ambas redes usaron capas apiladas de 128, 64, 32 y 16 unidades, activación tanh, Adam con tasa de 0.0001, batch de 64 y early stopping con paciencia de 5.

El hallazgo más útil para el proyecto es que el desempeño depende del largo de la ventana: con 6 horas de historia la GRU superó a la LSTM en MSE, MAE y MAPE, mientras que con 24 horas quedaron parejas. El autor lo atribuye a que la GRU no tiene una memoria separada, lo que la favorece en secuencias cortas. Los valores exactos solo se reportan en gráficas.

Mansur, Ramadhan y Bauravindah (2026) usan una GRU para predecir la llegada de pacientes a urgencias por hora (datos de 2014 a 2017 de un hospital). La arquitectura es mínima: una capa GRU de 50 unidades y una capa densa de salida, con Adam (tasa de 0.001), batch de 32, 50 épocas y una ventana deslizante de 30 días. Reportan un MAE de 10.61 y un MAPE de 9.23 %. Es evidencia débil, porque no comparan cuantitativamente contra otros modelos y usan un solo hospital.

En sentido contrario, en el trabajo de Zhao et al. (2019) ya revisado en la investigación de TCN, la GRU fue el peor de los tres modelos para flujo vehicular a corto plazo (MAE de 36.09, frente a 29.61 de la LSTM y 8.43 de la TCN).

5.3 GRU con datos de vacas y producción lechera

Wang et al. (2021) clasifican el comportamiento del ganado a partir de datos de acelerómetros en collares y aretes, usando cuatro conjuntos de datos. Compararon GRU y LSTM con distintas profundidades y anchos, y contra dos clasificadores convolucionales del estado del arte. Las redes recurrentes igualaron o superaron a las convolucionales con menos cómputo y memoria, y la GRU fue en general más precisa que la LSTM a pesar de ser más simple. La configuración que recomiendan por su balance entre precisión y complejidad es una sola capa GRU unidireccional de 64 unidades.

Naghashi, Boukadoum y Diallo (2025) proponen Bi-iGRU para pronosticar la producción lechera a lo largo de seis lactancias. Usaron 2.15 millones de registros de 29,136 vacas en 5,019 hatos de Quebec (2006–2017), con 19 variables que se convierten en 97 canales tras el one-hot encoding. La GRU se aplica de forma bidireccional sobre la dimensión de las variables (no sobre el tiempo) y la dinámica temporal se modela con capas densas. Promediando las lactancias 2 a 6 obtuvo un MSE de 0.0759 y un R^2 de 0.597, mejor que TimeMixer++, iTransformer y PatchTST, aunque por muy poco frente a S-Mamba y

MuMu+Attention. Es el trabajo más cercano al objetivo de ciclo de lactancia, pero sus datos son pruebas mensuales, no sesiones de ordeño en robot.

5.4 Parámetros propuestos

La siguiente configuración inicial se arma a partir de los valores de los estudios anteriores (selección propia, no probada todavía con nuestros datos):

- Ventana de entrada: 12 bloques (6 h) como base, comparando contra 48 bloques (24 h) para cubrir los horarios de alimentación de 7:30 y 14:30 (Das, 2023). Las ventanas se arman por robot (agrupando por ms), porque el dataset intercala los tres robots en orden de bloque_tiempo.
- Arquitectura base: GRU de 64 unidades, una capa, unidireccional (Wang et al., 2021), seguida de Dropout de 0.3, una capa densa de 32 con ReLU y una salida de 1 neurona con sigmoide, ya que pct_posible_cola va de 0 a 1.
- Variante más profunda: dos capas GRU de 64 (return_sequences=True) y 32 unidades, siguiendo la reducción progresiva de Das (2023) y de la LSTM de Ahmed et al. (2026).
- Optimizador: Adam con tasa de 0.001 (Mansur et al., 2026); bajar a 0.0001 si la pérdida es inestable (Das, 2023).
- Batch de 32 o 64 y hasta 100 épocas con early stopping sobre la pérdida de validación (paciencia de 5 a 10), restaurando los mejores pesos.
- Pérdida: MSE, calculada solo en los bloques con hubo_ordenios = 1 y objetivo no nulo.
- Preparación: StandardScaler para las variables numéricas y One-Hot Encoding para temporada. La validación usa el TimeSeriesSplit de 5 folds de BaseModeling, una partición temporal como la de Das (2023) y Zhao et al. (2019), cuidando que ninguna ventana cruce el límite entre entrenamiento y prueba.
- Métricas: MAE, RMSE y R^2 sobre pct_posible_cola, comparando contra una línea base simple.

5.5 Precauciones

- Resultados mixtos: la GRU ganó en Wang et al. (2021) y con ventanas cortas en Das (2023), pero perdió contra LSTM y TCN en Zhao et al. (2019). No hay una arquitectura ganadora de antemano.
- Interpretabilidad: la GRU es poco interpretable. La asociación con la etapa de lactancia debe probarse con pruebas estadísticas aparte.
- Sin precedente directo: no se encontraron estudios con GRU aplicados a colas de robots de ordeño. Los trabajos citados sirven como referencia de diseño, no de desempeño esperado.

6. Modelo MLP Classifier (Multi-layer Perceptron Classifier)

6.1 Introducción

El MLP Classifier es una red neuronal de tipo perceptrón multicapa que determina a qué clase pertenece cada registro. Dentro de este proyecto se utilizará para clasificar si en cada bloque de 30 minutos de los robots de ordeño del CAETEC hubo cola o no. Se eligió debido a que la cola depende de relaciones no lineales y de la variación a lo largo del día, por lo que un MLP se puede utilizar para aproximarlas sin necesidad de especificarlas de antemano (Hornik et al., 1989).

6.2 Arquitectura del MLP Classifier

Un MLP Classifier organiza sus neuronas en capas totalmente conectadas. Cada neurona calcula una suma ponderada de sus entradas, le suma un sesgo y posteriormente le aplica una función de activación.

Primero se tiene la capa de entrada, con una neurona por variable; en nuestro caso tendríamos 23 neuronas de entrada, o 26 al codificar la temporada. Posteriormente se tienen las capas ocultas, en donde se combinan las entradas de forma no lineal gracias a la función de activación; cabe mencionar que sin las capas ocultas la red sería una regresión logística. Para finalizar, se tiene la capa de salida, que es una sola neurona con función sigmoide, la cual en nuestro caso produciría la probabilidad de que haya cola en el bloque.

El MLP Classifier aprende minimizando la pérdida logarítmica (log-loss) más una penalización L2 sobre los pesos, la cual es controlada por alpha y en consecuencia reduce el

sobreajuste. En esta función, el primer término es el error de clasificación y el segundo es la regularización L2. El primer término castiga más al modelo mientras más seguro esté de una respuesta equivocada. Para reducir el valor de esta función se utiliza la retropropagación, la cual calcula cuánto contribuye cada peso al error propagándolo desde la salida hacia atrás (Rumelhart et al., 1986). Con esto, el optimizador Adam actualiza los pesos en cada época para que la pérdida disminuya.

En la Tabla 1 se mencionan los hiperparámetros principales en scikit-learn (Pedregosa et al., 2011):

Hiperparámetro	¿Qué controla?	Por defecto	Valores a probar
hidden_layer_sizes	Capas ocultas y neuronas	(100,)	(32,), (64, 32), (128, 64)
activation	Función de activación	relu	relu, tanh
alpha	Regularización L2	0.0001	0.0001, 0.001, 0.01
learning_rate_init	Tamaño del paso de actualización	0.001	0.001, 0.0001
max_iter	Número máximo de épocas	200	200, 500

Tabla 1 - Hiperparámetros a probar y su funcionamiento

6.3 Adaptación en el proyecto

Dentro de nuestro modelo se estima la variable hay_cola, la cual vale 1 cuando más del 50 % de los ordeños del bloque tuvo posible cola y 0 en el caso contrario. Esto se cuenta en 132,028 bloques de 30 minutos repartidos en los tres robots, los cuales van de agosto de 2022 a diciembre de 2025. De hecho, las redes neuronales ya han sido aplicadas a los datos de sistemas de ordeño automático, sobre todo para detectar mastitis y predecir la producción de leche (Fuentes et al., 2020; Ozella et al., 2023); en nuestro caso se aplicaría a la formación de colas.

Para la utilización del MLP Classifier se realizará un pipeline en donde se escalarán las variables numéricas utilizando StandardScaler, posteriormente se codificará la temporada y finalmente se entrenará el modelo. Se validará con un TimeSeriesSplit de 5 folds, en el cual cada fold se entrena con el pasado y prueba con el siguiente periodo, debido a que una validación aleatoria sobreestima el desempeño en los datos temporales (Bergmeir & Benítez, 2012). Los valores de la Tabla 1 se buscarán con GridSearchCV sobre esos mismos folds; luego se medirán el F1-score, la precisión y el recall, y se compararán contra dos referencias sin modelo, suponiendo que la cola es igual a la del bloque anterior. Para finalizar, se estimará la importancia por permutación, en donde se desordena una variable y se mide cuánto empeoró el F1-score (Fisher et al., 2019).

6.4 Precauciones

- Fuga de información: el escalado se ajusta solo con el entrenamiento de cada fold, por eso va dentro del pipeline. Para predecir se usan los valores de ocupación del bloque anterior, no los del mismo bloque, los cuales todavía no se conocen (Kaufman et al., 2012).
- Early stopping: se deja en False, debido a que separa una validación al azar que mezcla diferentes periodos.
- Balance de clases: se revisa qué proporción de los bloques tiene cola; si alguna clase es más frecuente obtendrá un F1-score más alto, por lo que se compara con las referencias sin modelo.
- Convergencia y semilla: se revisa la curva de pérdida, se fija el random_state y se reporta el promedio y la desviación de las métricas en los 5 folds.
- Cambios en el tiempo: los ordeños por robot se triplicaron entre 2022 y 2025, por lo que se revisan los resultados de cada fold y no solo el promedio.
- Expectativa realista: en los datos tabulares los modelos de árboles suelen igualar o superar a las redes neuronales (Shwartz-Ziv & Armon, 2022), por lo que si el MLP queda por debajo es un resultado válido que se reporta.

7. Modelos ARIMA y SARIMA

7.1 Fundamentos de los modelos

ARIMA (AutoRegressive Integrated Moving Average) modela una serie temporal a partir de su dependencia con el pasado. Se representa como ARIMA(p, d, q): p indica los rezagos autorregresivos, d las diferenciaciones y q los términos de errores anteriores. La media móvil se refiere a errores del modelo, no a un promedio móvil usado para suavizar los datos. Si d = 0, el modelo corresponde a un ARMA.

SARIMA agrega una estructura estacional y se escribe ARIMA(p, d, q)(P, D, Q)s. Los parámetros P, D y Q representan componentes autorregresivos, diferenciación y errores estacionales; s es el número de observaciones por ciclo. La diferenciación busca una serie compatible con estacionariedad; no debe aplicarse automáticamente por el solo hecho de que exista un patrón diario.

7.2 Dataset y variable objetivo

Se propone utilizar el dataset final para serie de tiempo, donde cada robot conserva una malla de 30 minutos. Se ajustará un modelo por robot, ordenado por bloque_tiempo, ya que intercalar los tres robots como una sola serie introduciría rezagos sin significado temporal.

La variable objetivo será y = pct_posible_cola, entre 0 y 1. Es la proporción de ordeños cuyo hueco hasta el siguiente ordeño del robot es de 0 a 2 minutos. Se pronosticará este indicador agregado, el cual no mide directamente la longitud de la cola, los minutos de espera ni la probabilidad individual de que una vaca espere. El dataset de regresión no se utilizará directamente para ARIMA/SARIMA porque contiene huecos entre bloques.

7.3 Justificación de la selección

En la Tabla 2 se resumen las razones para incluir estos modelos:

Razón	Relación con el proyecto
Dependencia temporal	Permite comprobar si la historia de posible cola de cada robot aporta información para anticipar el siguiente bloque.

Patrones recurrentes	La malla permite explorar un ciclo diario de 48 bloques y uno semanal de 336. Son hipótesis que deben revisarse en entrenamiento, no patrones ya demostrados.
Referencia estadística	ARIMA permite medir cuánto aporta la dinámica reciente. SARIMA permite evaluar el beneficio adicional de incorporar un ciclo estacional.
Modelo parsimonioso	Ofrece una alternativa con estructura explícita de rezagos y errores para comparar con MLP, LSTM, GRU y TCN. Sus coeficientes no prueban causalidad.

Tabla 2 - Justificación de ARIMA y SARIMA

7.4 Posible extensión: SARIMAX

ARIMA y SARIMA univariados utilizan la historia de y. Si se añaden predictores externos, la variante será una regresión con errores ARIMA/SARIMA, denominada SARIMAX. Para iniciar, se proponen variables de calendario y min_desde_alimento calculadas para el bloque futuro, pues se conocen por anticipado bajo el horario programado. Las variables operativas solo se incorporarían con rezagos realmente disponibles al emitir el pronóstico.

Las proporciones de lactancia del bloque futuro no se conocen de antemano. Sus rezagos podrían evaluarse como una extensión, pero ARIMA y SARIMA univariados no explican por sí solos qué etapa de lactancia se asocia con las colas.

7.5 Configuración y propuestas iniciales

Las siguientes decisiones son propuestas iniciales del proyecto. Deben ajustarse tras analizar las series reales; no son parámetros óptimos obtenidos experimentalmente.

Elemento	Propuesta inicial
Horizonte	Un bloque futuro (30 minutos). Como experimento adicional, cuatro bloques (2 horas), reportando el error por horizonte.

ARIMA de referencia	Comparar órdenes pequeños con p y q entre 0 y 2, y d entre 0 y 1 según el diagnóstico. Incluir ARIMA(1,0,0) y ARIMA(1,0,1) entre los candidatos.
SARIMA candidato	Evaluar s = 48 y órdenes estacionales P y Q entre 0 y 1; D entre 0 y 1 solo si el diagnóstico lo justifica. SARIMA(1,0,1)(1,0,0) con s = 48 es un candidato inicial, no una selección final.
Presupuesto de búsqueda	Preseleccionar de 8 a 12 configuraciones por robot. Evitar el producto cartesiano exhaustivo de todos los órdenes y medir convergencia, memoria y tiempo.
Estacionalidad semanal	Explorar s = 336 únicamente si hay evidencia y recursos. Como alternativa, evaluar el calendario semanal o términos de Fourier mediante regresión con errores ARIMA.
Diagnóstico	Revisar gráficas, ACF/PACF y pruebas ADF/KPSS en entrenamiento para orientar d. ADF contrasta raíz unitaria y KPSS contrasta estacionariedad. No determinan por sí solas D.

Tabla 3 - Configuración inicial de ARIMA y SARIMA

7.6 Validación temporal y referencias

Se reservará el tramo final como prueba intocable. En el periodo anterior se realizarán cinco cortes cronológicos con entrenamiento creciente y fechas comunes para los tres robots. En cada corte se estimará con el pasado y se evaluará con origen móvil al horizonte definido. Se actualizará el estado solo cuando las observaciones estén disponibles.

Se comparará con la persistencia (última etiqueta disponible) y con el valor de la misma franja del día anterior. AIC/AICc servirá como apoyo entre candidatos comparables, con la misma respuesta y diferenciación; la selección principal dependerá del error fuera de muestra. Se reportarán MAE y RMSE por robot, periodo y horizonte, y R^2 como complemento. Un MAE de 0.05 equivale a 5 puntos porcentuales del indicador. No se usará MAPE debido a los ceros (Hyndman & Athanasopoulos, 2021).

7.7 Precauciones

- Ausencia de ordeños: hay 47,792 bloques sin ordeños, con objetivo igual a 0 por construcción. Esos ceros no demuestran ausencia real de cola ni descartan fallos de registro. Se conservará la malla y se reportará el desempeño tanto en el conjunto completo como en los bloques con hubo_ordenios = 1. Esta selección es un análisis retrospectivo, no una variable conocida del futuro. Las interrupciones confirmadas se tratarán como faltantes, sin comprimir el calendario.
- Predictores futuros: no se introducirán total_ordenios, duracion_total_min, pct_ocupacion_robot ni ordenios_otros_robots del bloque que se quiere pronosticar. Tampoco vacas_activas_dia calculada sobre el día completo antes de que termine. En SARIMAX se requieren valores de las entradas para los periodos futuros; disponer de esas columnas en el archivo histórico no significa que puedan usarse en operación.

7.8 Decisión de modelado

Se evaluará ARIMA como referencia temporal y SARIMA como candidato principal si se confirma una estructura diaria útil. Inicialmente no se utilizará SARIMAX, ya que el alcance de ARIMA y SARIMA se limitará a evaluar patrones temporales de posible cola, como la dependencia entre bloques y las variaciones recurrentes a lo largo del día. Su utilidad predictiva se evaluará mediante la comparación con una referencia simple fuera de muestra. Las asociaciones con las etapas de lactancia se estudiarán por separado mediante modelos de regresión. Estos enfoques serán complementarios y no sustituirán otras tareas de modelado.

8. Comparación de las técnicas

La Tabla 4 resume cómo se plantea cada técnica con nuestros datos.

Técnica	Dataset y variable objetivo	Configuración inicial	Evaluación
LSTM	Serie de tiempo (requiere datos continuos); variable objetivo por definir	Referencia: capas de 128, 64 y 32 unidades con Dropout de 0.5 (Ahmed et al., 2026)	Por definir

TCN	Serie de tiempo; bloque t con ventana hasta t−1	Kernel de 3 y cuatro niveles de dilatación (campo receptivo de 61 bloques)	Comparar contra la LSTM y una línea base
GRU	Serie de tiempo; pct_posible_cola (regresión)	Ventana de 12 bloques, GRU de 64 unidades, salida sigmoide y MSE con máscara	MAE, RMSE y R^2; TimeSeriesSplit de 5 folds
MLP Classifier	Regresión; hay_cola (clasificación, más de 50 %)	Grid Search de capas ocultas, activación, alpha y tasa de aprendizaje	F1, precisión y recall; TimeSeriesSplit de 5 folds
ARIMA / SARIMA	Serie de tiempo, un modelo por robot; pct_posible_cola	SARIMA(1,0,1)(1,0,0) con s = 48 como candidato inicial	MAE, RMSE y R^2; cinco cortes y prueba final

Tabla 4 - Comparación de las técnicas investigadas

10. Conclusiones

Las cinco técnicas son complementarias. ARIMA/SARIMA funciona como referencia estadística de la dinámica temporal de cada robot; la LSTM, la GRU y la TCN pueden aprovechar secuencias más largas y varias variables a la vez; y el MLP Classifier evalúa si las variables del bloque, sin estructura secuencial, bastan para anticipar la cola. Ninguna de las investigaciones encontró antecedentes de estas técnicas aplicadas directamente a colas en robots de ordeño, por lo que los estudios citados sirven como referencia de diseño y no como desempeño esperado. El siguiente paso es acordar la variable objetivo, la línea base y las fechas de validación comunes, y entrenar los modelos con esos mismos criterios para elegir el que mejor equilibre el error de predicción y la utilidad para entender la formación de colas.

11. Referencias

Ahmed, G., Malick, R. A. S., Al-Shamayleh, A. S., et al. (2026). STELLAR-CB: Synthetic temporal LSTM for livestock activity recognition—cow behaviour. Veterinary Medicine and Science, 12(4), e70827. https://doi.org/10.1002/vms3.70827

Amat Rodrigo, J. (s. f.). Deep learning for time series prediction: Recurrent neural networks (RNN), gated recurrent unit (GRU), and long short-term memory (LSTM). Ciencia de Datos. https://dev.cienciadedatos.net/documentos/py54-forecasting-with-deep-learning

Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. https://doi.org/10.48550/arXiv.1803.01271

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. Information Sciences, 191, 192–213. https://doi.org/10.1016/j.ins.2011.12.028

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. Proceedings of EMNLP 2014, 1724–1734. https://doi.org/10.3115/v1/D14-1179

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv:1412.3555. https://doi.org/10.48550/arXiv.1412.3555

Das, L. C. (2023). Traffic volume prediction using memory-based recurrent neural networks: A comparative analysis of LSTM and GRU. arXiv:2303.12643. https://doi.org/10.48550/arXiv.2303.12643

Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. Journal of Machine Learning Research, 20(177), 1–81.

Fuentes, S., Gonzalez Viejo, C., Cullen, B., Tongson, E., Chauhan, S. S., & Dunshea, F. R. (2020). Artificial intelligence applied to a robotic dairy farm to model milk productivity and quality based on cow data and daily environmental parameters. Sensors, 20(10), 2975. https://doi.org/10.3390/s20102975

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3.ª ed.). OTexts. https://otexts.com/fpp3/

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), Art. 15. https://doi.org/10.1145/2382577.2382579

Mansur, A., Ramadhan, F. A., & Bauravindah, A. (2026). GRU-based forecasting of emergency department patient volume for hospital operations and supply chain planning. Mathematical Modelling of Engineering Problems, 13(6), 565–574. https://doi.org/10.18280/mmep.130311

Naghashi, V., Boukadoum, M., & Diallo, A. B. (2025). A bi-directional cross-channel RNN model for time-series forecasting of dairy production. Scientific Reports. https://doi.org/10.1038/s41598-025-28864-z

Ozella, L., Brotto Rebuli, K., Forte, C., & Giacobini, M. (2023). A literature review of modeling approaches applied to data collected in automatic milking systems. Animals, 13(12), 1916. https://doi.org/10.3390/ani13121916

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Rahman, R., & Hasan, S. (2021). Real-time signal queue length prediction using long short-term memory neural network. Neural Computing and Applications, 33(7), 2821–2834. https://doi.org/10.1007/s00521-020-05196-9

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533–536. https://doi.org/10.1038/323533a0

Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. Information Fusion, 81, 84–90. https://doi.org/10.1016/j.inffus.2021.11.011

statsmodels. (s. f.). SARIMAX: Seasonal autoregressive integrated moving-average with exogenous regressors. https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html

Wang, L., Arablouei, R., Alvarenga, F. A. P., & Bishop-Hurley, G. J. (2021). Animal behavior classification via accelerometry data and recurrent neural networks. arXiv:2111.12843. https://doi.org/10.48550/arXiv.2111.12843

Wei, Y., Zhang, H., Gong, C., Wang, D., Ye, M., & Jia, Y. (2023). Study of pose estimation based on spatio-temporal characteristics of cow skeleton. Agriculture, 13(8), 1535. https://doi.org/10.3390/agriculture13081535

Zhao, W., Gao, Y., Ji, T., Wan, X., Ye, F., & Bai, G. (2019). Deep temporal convolutional networks for short-term traffic flow forecasting. IEEE Access, 7, 114496–114507. https://doi.org/10.1109/ACCESS.2019.2935504